

Architectural Optimization and Best Practices for High Performance Computing on AWS

The evolution of High Performance Computing (HPC) from isolated, on-premises supercomputing centers to the elastic, distributed environment of the cloud represents a transformative shift in computational science and engineering. Historically, HPC was defined by massive capital expenditures, rigid hardware refresh cycles, and the physical constraints of data center power and cooling. The AWS Well-Architected High Performance Computing (HPC) Lens provides a structured methodology to transcend these traditional limitations, enabling organizations to build secure, high-performing, resilient, and efficient infrastructures.¹ By applying the six pillars of the Well-Architected Framework—Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, and Sustainability—architects can design systems that not only match the performance of legacy clusters but also offer superior flexibility and time-to-results.³

Foundational Scenarios and Design Principles

The architecture of an HPC system is fundamentally dictated by the communication patterns and data dependencies of the applications it supports. The HPC Lens categorizes these into three primary scenarios: loosely coupled, tightly coupled, and hybrid architectures.²

Loosely Coupled Architectures

Loosely coupled workloads, often described as "embarrassingly parallel" or "high-throughput computing," consist of independent tasks that do not require inter-process communication during execution.⁴ These workloads typically rely on data parallelism, where a massive dataset is partitioned into smaller segments, each processed by a separate compute instance.⁴ Because the failure of a single node does not jeopardize the entire job, these architectures are highly resilient and ideally suited for the cost-saving potential of Amazon EC2 Spot Instances.⁶

A common architectural pattern for loosely coupled workloads involves the use of AWS Batch or a serverless approach using AWS Lambda.⁵ In a typical bioinformatics pipeline, for example, thousands of genomic sequences might be processed simultaneously. The completion of the overall job depends only on the aggregate throughput of the system. Consequently, the architect prioritizes horizontal scalability over low-latency networking.⁴

Tightly Coupled Architectures

Tightly coupled workloads involve parallel processes that must frequently exchange information, often at every iteration of a mathematical solver.⁴ These applications, such as

Computational Fluid Dynamics (CFD), finite element analysis (FEA), and weather forecasting, typically utilize the Message Passing Interface (MPI) to synchronize state across thousands of processor cores.⁴ The performance of these systems is highly sensitive to "jitter" and network latency; a single slow node or a congested network path can stall the entire simulation.¹⁰

To address these requirements, tightly coupled architectures necessitate specialized hardware configurations, including the Elastic Fabric Adapter (EFA) and Cluster Placement Groups, to ensure sub-millisecond latencies and high-bandwidth interconnects.⁹ The reliance on synchronized communication often dictates the use of homogeneous compute instances to prevent performance imbalances.¹³

Hybrid and Cloud-Bursting Architectures

Hybrid scenarios involve extending on-premises HPC capabilities into the AWS Cloud to handle peak demand, a strategy known as "cloud bursting".⁵ Tools like AWS ParallelCluster facilitate this by allowing the same job scheduler (such as Slurm) to manage both on-premises nodes and cloud-based EC2 instances.⁸ This architectural flexibility allows organizations to maintain steady-state workloads locally while leveraging the cloud's elasticity for large-scale experiments or urgent projects.⁸

Feature	Loosely Coupled	Tightly Coupled	Hybrid / Bursting
Communication	None or minimal ⁴	Frequent, high-bandwidth ⁴	Varies by job ⁸
Network	Standard TCP/IP ⁴	EFA / SRD ¹⁰	VPN / Direct Connect ¹⁶
Resilience	High (Spot-friendly) ⁶	Low (checkpointing needed) ¹⁷	Mixed ⁸
Scheduler	AWS Batch / Lambda ⁵	Slurm / AWS PCS ¹⁸	Slurm ¹⁴
Scaling	Horizontal ⁶	Parallel (Fixed Cluster) ⁹	Elastic expansion ¹⁵

Overarching Design Principles

The HPC Lens introduces specific design principles that challenge traditional supercomputing assumptions. Architects are encouraged to move away from static, "always-on" clusters toward dynamic architectures that grow and shrink in response to demand.¹⁵ By aligning the procurement model—such as using a combination of On-Demand, Savings Plans, and Spot

Instances—to the specific demand profile of the workload, organizations can optimize for both cost and performance.¹⁵

Furthermore, the lens emphasizes the importance of data locality and secure collaboration. Rather than transferring massive datasets to researchers, architectures should leverage remote visualization technologies like NICE DCV to bring the researcher to the data.⁵ Automation via Infrastructure as Code (IaC) is also foundational, ensuring that complex HPC environments are reproducible and free from the configuration drift common in manual setups.¹⁵

Performance Efficiency: Networking and Compute Optimization

Performance efficiency in HPC is characterized by the ability to utilize computing resources to meet system requirements at scale. This involves deep technical integration of networking protocols, compute instance selection, and storage performance.²⁰

Networking: The Elastic Fabric Adapter and SRD Protocol

For tightly coupled simulations, the network often becomes the primary bottleneck. Standard Ethernet and TCP/IP stacks introduce significant latency and jitter due to kernel overhead and congestion control mechanisms.¹⁰ The Elastic Fabric Adapter (EFA) is a specialized network interface for Amazon EC2 that enables kernel-bypass networking, allowing applications to communicate directly with the network hardware.¹⁰

The underlying technology of EFA is the Scalable Reliable Datagram (SRD) protocol. SRD was developed by AWS specifically to overcome the limitations of traditional networking in hyper-scale data centers.¹¹ Unlike InfiniBand or TCP, which rely on single-path routing and strict packet ordering, SRD utilizes a multi-path approach.¹¹

1. **Multi-Path Routing:** SRD sprays packets across as many as ⁶⁴ separate network paths. This prevents any single congested link from impacting the entire flow, resulting in more consistent and lower tail latencies.¹¹
2. **Out-of-Order Delivery:** Because SRD delivers packets over multiple paths, they may arrive at the destination out of order. The reordering is handled by the messaging layer (such as Libfabric) rather than the transport layer, which eliminates the "head-of-line blocking" common in TCP.¹⁰
3. **Hardware-Based Reliability:** SRD is implemented in the custom AWS Nitro networking card. This hardware implementation allows for microsecond-level detection of network fluctuations and immediate retransmission of dropped packets, bypassing the slower response times of a software-based OS kernel.¹¹

For the architect, the practical implication is that EFA-enabled instances must be launched into a **Cluster Placement Group**. This ensures that the instances are physically located in the same

network segment within a single Availability Zone, providing the necessary proximity to leverage the single-digit microsecond latencies offered by EFA.⁹

Compute Architecture and Instance Selection

The selection of EC2 instance types for HPC requires a nuanced understanding of the application's computational profile. AWS provides several instance families optimized specifically for HPC workloads.⁹

- **HPC-Optimized Families:** Instances such as hpc6a (powered by AMD EPYC processors) and hpc7g (powered by AWS Graviton) are designed for compute-intensive workloads that require high memory bandwidth and EFA networking.⁹
- **High-Memory and Storage Instances:** For workloads like in-memory databases or large-scale data analytics, memory-optimized instances (such as the r family) provide the necessary capacity to avoid disk I/O bottlenecks.²¹
- **Accelerated Computing:** For machine learning, seismic processing, and molecular dynamics, GPU-accelerated instances (such as the p4d or p5 families) provide massive parallel processing power.⁹ These instances often support GPUDirect Storage (GDS), which allows for a direct data path between GPU memory and the file system, bypassing the CPU to further reduce latency.²⁵

Architects should prioritize virtualized instances over bare-metal options as a standard starting point. The modern AWS Nitro System provides a lightweight hypervisor that delivers performance nearly indistinguishable from bare-metal while offering the management and elasticity benefits of virtualization.²¹

Instance Family	Processor Type	Primary Use Case	Network Bandwidth
hpc6a	AMD EPYC	General HPC / CFD / FEA ⁹	100 Gbps (EFA) ⁹
hpc7g	AWS Graviton	Sustainable HPC / Weather ⁹	200 Gbps (EFA) ⁹
p4d	NVIDIA A100	ML / Deep Learning ⁹	400 Gbps (EFA) ⁹
c7g	AWS Graviton	Compute-intensive workloads ²⁴	25 Gbps ²⁴
m6i	Intel Xeon	Balanced General	Up to 50 Gbps ⁹

		Purpose ¹⁴	
--	--	-----------------------	--

Storage Performance: Parallel File Systems and S3 Integration

HPC applications typically involve massive I/O requirements, where thousands of compute nodes must simultaneously read from and write to a shared storage layer. Traditional Network Attached Storage (NAS) solutions often fail at this scale due to metadata bottlenecks and single-controller limitations.²⁸

Amazon FSx for Lustre

Amazon FSx for Lustre is the primary storage solution recommended for HPC on AWS. Based on the popular Lustre open-source parallel file system, FSx for Lustre is designed to scale throughput and IOPS linearly with the size of the file system.²⁸ In a Lustre architecture, data is "striped" across multiple storage servers and disks, allowing clients to access data in parallel without contending for a single communication path.²⁸

Architects can choose between different deployment types based on the workload's durability requirements. **Scratch** file systems are intended for temporary processing and do not replicate data, offering high performance at the lowest possible cost.²⁸ **Persistent** file systems are designed for longer-term storage and automatically replicate data across different servers to ensure high availability.²⁸

Data Repository Association (DRA)

A critical feature of FSx for Lustre is its native integration with Amazon S3 through the Data Repository Association (DRA).²⁹ DRA allows the architect to link a Lustre file system to an S3 bucket, effectively using S3 as a long-term data lake and Lustre as a high-performance cache.²⁹

- **Metadata Syncing:** When a link is established, FSx for Lustre imports the metadata (file names, sizes, permissions) from S3. The actual data is only pulled into Lustre when a file is first accessed by a compute node, a process known as "lazy loading".²⁸
- **Automatic Export:** Results generated during a simulation can be automatically exported back to S3, ensuring that valuable data is durably stored even after the file system is deleted.²⁸
- **Data Release:** To manage the capacity of the Lustre file system, the architect can use "data repository tasks" to release the local copies of files that have not been accessed recently. This frees up high-performance storage while keeping the master copy safe in S3.²⁹

Cost-Optimized Storage Tiers

For workloads with diverse access patterns, the **Intelligent-Tiering** storage class for FSx for

Lustre provides a mechanism to reduce costs by automatically moving data between storage tiers.²⁹ Data that hasn't been accessed for **30** days is moved to an Infrequent Access tier, and data unused for **90** days moves to an Archive tier, resulting in up to a **96%** reduction in storage costs compared to the Frequent Access tier.²⁹ This tiering happens transparently to the application, maintaining sub-millisecond latencies for any data currently residing in the SSD read cache.²⁹

Operational Excellence and Compute Orchestration

Operational excellence in HPC involves the automation of cluster management and the implementation of robust monitoring to ensure that computational resources are used effectively.¹

Orchestration Choice: Parallel Computing Service vs. ParallelCluster vs. Batch

The choice of orchestration tool is one of the most significant decisions for an HPC architect, as it determines the operational burden and the user experience for researchers.⁸

1. **AWS Parallel Computing Service (PCS):** PCS is a fully managed service that provides a cloud-native Slurm environment. AWS handles the management, patching, and scaling of the Slurm scheduler and head nodes.¹⁸ This is ideal for production environments where operational uptime and compliance are paramount, as it removes the "heavy lifting" of maintaining the cluster infrastructure.¹⁸
2. **AWS ParallelCluster:** ParallelCluster is an open-source tool that automates the deployment of HPC clusters using AWS CloudFormation.¹⁴ It gives the architect full control over the configuration, including custom AMIs and post-install scripts.⁸ It is the preferred choice for research labs that need to tinker with the environment or for "cloud bursting" scenarios that require a specific version of Slurm to match an on-premises setup.⁵
3. **AWS Batch:** Batch is a fully managed job scheduler optimized for containerized, loosely coupled workloads.⁶ It dynamically provisions compute resources—either EC2 or Fargate—based on the requirements of the jobs in the queue.⁵ Batch is ideal for pipelines that integrate with other AWS services like Step Functions or EventBridge.⁸

Criterion	AWS PCS	AWS ParallelCluster	AWS Batch
Management	Fully Managed by AWS ¹⁸	Self-Managed via Toolkit ¹⁸	Fully Managed Scheduler ⁶

Interface	AWS Console / SDK / CLI ¹⁸	Custom CLI / UI ¹⁸	AWS Console / CLI / JSON ⁶
Customization	High (Slurm settings) ¹⁸	Extreme (Full Stack Control) ¹⁸	Medium (Container Focus) ⁶
SLA	Managed AWS SLA ¹⁸	No Managed SLA ¹⁸	Managed AWS SLA ⁶
Deployment	"HPC as a Service" ¹⁸	CloudFormation-based ¹⁴	Cloud-native ⁶

Monitoring and Observability

A well-architected HPC system must provide visibility into both infrastructure health and application performance. Standard metrics like CPU and memory utilization from CloudWatch are necessary but often insufficient for HPC.¹⁹ Architects should implement custom metrics to track application-specific progress (e.g., iterations per hour) and network health (e.g., SRD retransmission rates).³¹ This data allows the organization to identify bottlenecks and right-size resources effectively, ensuring that budget is spent on compute power rather than waiting for I/O.¹⁵

Reliability and Failure Management

Reliability in HPC is defined by the system's ability to recover from failures and perform its intended functions under changing requirements.¹

Responding to Hardware and Infrastructure Failure

In the massive scale of HPC, hardware failure is an statistical inevitability rather than a possibility. Tightly coupled jobs are particularly vulnerable; a single failed node in a 1,000-node cluster can cause the entire MPI job to fail.⁹

- **Checkpointing Strategy:** To mitigate the impact of failure, applications should implement periodic checkpointing. By saving the simulation's state to persistent storage (like FSx for Lustre or S3) every few hours, a job can be resumed from the last checkpoint rather than restarting from zero.⁷
- **Self-Healing Infrastructure:** Schedulers like AWS Batch and Slurm (via PCS or ParallelCluster) automatically monitor the health of compute nodes. If a node becomes unresponsive, the scheduler can terminate it, provision a replacement, and re-queue the affected task.⁶

Managing Spot Interruption

When using Spot Instances for loosely coupled workloads, the architect must plan for the two-minute interruption notice provided by AWS. Schedulers should be configured to gracefully stop current tasks and save progress when this notice is received.⁶ For long-running loosely coupled tasks, AWS Batch can automatically handle the re-scheduling of interrupted jobs, often on different instance types to ensure job completion.⁶

Security and Governance in Research Environments

The security pillar focuses on protecting information and systems through confidentiality, integrity, and robust identity management.¹

Infrastructure Protection and Data Privacy

HPC environments often process sensitive proprietary or regulated data.

- **VPC and Network Isolation:** All compute nodes should reside in private subnets, with communication limited by strict Security Group rules. For EFA, the Security Group must allow all-protocol traffic within the group to support the low-level communication required by SRD.⁹
- **Encryption:** Encryption should be enabled at rest using AWS KMS keys for EBS volumes, FSx file systems, and S3 buckets.¹² Nitro-based instances also provide transparent encryption of data in transit across the VPC network.¹²
- **Identity Management:** Use IAM roles and instance profiles to grant compute nodes temporary, scoped permissions. This avoids the risk of embedding static credentials within application code or AMIs.¹²

Secure Collaboration and Remote Access

Scientific research is a collaborative endeavor, often requiring data sharing across institutions. The HPC Lens recommends leveraging the fine-grained access control of Amazon S3 to grant cross-account access to datasets.¹⁵

For researchers requiring interactive access to results, remote visualization via NICE DCV is the preferred architectural pattern. DCV uses a high-performance streaming protocol that delivers only the rendered pixels to the researcher's client. This ensures that the raw data never leaves the secure AWS environment, facilitating collaboration while maintaining strict data egress controls.⁵

Cost Optimization: Maximizing Return on Research

Cost optimization in HPC is not merely about minimizing spend; it is about maximizing the computational output for every dollar spent.³³

Procurement Strategy and FinOps

A mature HPC architecture utilizes a multi-tiered procurement strategy to balance cost and

availability.¹⁵

- **Baseline Capacity:** Use Savings Plans or Reserved Instances for the "head node" and the minimum number of compute nodes required for constant, daily throughput.¹⁵
- **Burst Capacity:** Use On-Demand instances for time-critical projects where immediate availability is worth the premium.⁹
- **Scale Capacity:** Use Spot Instances for large, loosely coupled pipelines that can tolerate interruptions, achieving up to **90%** savings.⁶

Rightsizing and Decommissioning

- **Instance Selection:** Avoid over-provisioning memory or CPU. If a simulation only uses **32** GB of RAM, launching it on an instance with **256** GB of RAM is wasteful. Tools like AWS Compute Optimizer can provide data-driven recommendations for rightsizing.¹⁹
- **Automated Lifecycle:** Ensure that clusters are terminated automatically once a project is complete. ParallelCluster and AWS PCS facilitate this by scaling down the number of nodes to zero when the queue is idle.¹⁴
- **Tagging and Chargeback:** Implement a robust tagging policy to attribute costs to specific researchers, projects, or grants. This visibility encourages a cost-aware culture and allows for more accurate budgeting of future research projects.¹⁹

Sustainability and the Carbon Footprint of Computation

As HPC workloads are highly energy-intensive, the sustainability pillar focuses on minimizing the environmental impact of these operations through high utilization and efficient technology.¹

Architectural Efficiency and Hardware Choice

- **Processor Efficiency:** Favor AWS Graviton-based instances (such as hpc7g) when possible. Graviton processors are designed for high performance and low power consumption, offering superior performance-per-watt compared to traditional x86 architectures.⁵
- **High Utilization:** Designing for elasticity ensures that hardware is not idling. A cluster that scales to zero when not in use is fundamentally more sustainable than a fixed-size on-premises cluster that runs at low utilization during off-peak hours.⁷

Data and Software Optimization

- **Reducing Data Movement:** Minimize the transfer of data between Regions or across the internet. By processing data in the same Region where it is stored, organizations reduce both cost and energy consumption.³⁵

- **Application Tuning:** Code that runs more efficiently uses less electricity. Profiling and optimizing MPI communication or using specialized hardware libraries (like NCCL for machine learning) directly contributes to the sustainability of the workload.¹⁷

Summary for the Solutions Architect

The High Performance Computing Lens provides a blueprint for building sophisticated computational environments that leverage the best of the AWS Cloud. For the Solutions Architect, several key technical themes emerge as critical for both project success and certification readiness:

1. **Networking is the Interconnect:** For tightly coupled workloads, EFA and the SRD protocol are non-negotiable. Understanding how SRD's multi-pathing and hardware implementation solve the problem of jitter is essential.¹¹
2. **Storage as a Performance Lever:** FSx for Lustre is more than a file system; it is a high-performance data bridge to S3 via DRA. The ability to scale throughput linearly and use Intelligent-Tiering to manage costs is a central architectural advantage.²⁸
3. **The Shift to Managed Services:** The introduction of AWS PCS marks a move toward "HPC as a Service," allowing teams to focus on the science rather than the scheduler, while ParallelCluster remains the tool for high-customization research.¹⁸
4. **Resilience through Architecture:** Reliability in HPC is achieved through a combination of application-level checkpointing and cloud-native features like automatic node replacement and Spot interruption handling.⁶

By integrating these principles into their designs, architects can ensure that their HPC workloads are not only high-performing but also secure, cost-effective, and sustainable, enabling the next generation of scientific and engineering breakthroughs.